

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based on your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
In [3]: import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1
input_string = "Babak"
result = hash_to_range(input_string)
print(result)
```

3

► Question 1

Starter Code for Question 1

```
In [ ]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def is_duplicate(root: TreeNode) -> int:
    # TODO
```

► Question 2

Starter Code for Question 2

```
In [ ]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
    # TODO
```

► Question 3

Starter Code for Question 3

```
In [1]: from typing import List

def missing_num(nums: List[int]) -> List[int] or int:
    n = len(nums)
    full_set = set(range(n + 1))
    nums_set = set(nums)
    missing = sorted(list(full_set - nums_set))
    return missing if missing else -1
```

```
In [2]: test = [6, 8, 2, 3, 5, 7, 0, 1, 10]
print(missing_num(test))
```

[4, 9]

Part 2:

You and your partner must share each other's Assignment 1 submission.

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

The goal of example 2 in assignment 1 is to define if the sequence of brackets match.

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

```
In [ ]: assert is_valid_brackets("([{}])") == False

# This should result in a False value because the brackets in the sequence
# do not match each other.
```

- Copy the solution your partner wrote.

```
In [ ]: brackets = {'(': ')', '[': ']', '{': '}'
opening_brackets = brackets.values()
closing_brackets = brackets.keys()

def is_valid_brackets(s: str) -> bool:
    stack = []
    for char in s:
        if char in opening_brackets:
            stack.append(char)
        elif char in closing_brackets:
            if len(stack) == 0 or stack.pop() != brackets[char]:
                return False
        else:
            raise ValueError(f"Invalid character: '{char}'")
    return len(stack) == 0
```

- Explain why their solution works in your own words.

My partners solution works because they created a helper data structure called 'brackets' which defines all possible brackets. They then iterate a function which returns an empty stack if the brackets match. If there is a return stack with unopen brackets it returns false which means there is not a matching set of brackets.

- Explain the problem's time and space complexity in your own words.

The Time complexity explained by my partner is valid. We see $O(n)$, because the function iterates over the string of brackets of length n only once.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

One potential improvement would be to adjust 'opening_brackets' and 'closing_brackets'. `brackets.values()` and `brackets.keys()` return view objects, which are less efficient to search in repeatedly.

Instead evaluating as a set could make lookups faster than using dict_values

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

For Assignment 1, I was tasked with solving a problem that required moving all zero values in a list to the end, while preserving the order of the non-zero elements. I approached this by iterating through the list once to identify and move the non-zero values to the front. Then, I filled the remaining positions with zeros. This approach maintained the relative order of non-zero elements and achieved an efficient solution.

The logic was implemented with a single pass through the list using a pointer to track the index of the last non-zero value inserted. This resulted in an $O(n)$ time complexity, where n is the length of the list, since each element was visited at most twice — once for reading and once for writing. This was a significant improvement over naïve approaches that might involve multiple passes or unnecessary swaps.

During the peer review of my partner's assignment 1, I explained my thought process, walked through the code, and answered questions about the logic and performance. Reviewing their solution helped me reflect on alternative methods and reinforced the importance of writing clean, readable code. The collaborative feedback was helpful for identifying small optimizations and improvements in explanation clarity regarding optimizations that could theoretically iterate faster over longer strings.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🔔 Please review our [Assignment Submission Guide](#) 🔔 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (`assignment_2.ipynb`) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☒ Created a branch with the correct naming convention.
- ☒ Ensured that the repository is public.
- ☒ Reviewed the PR description guidelines and adhered to them.
- ☒ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

